

Guía de Laboratorio #6: Adquisición de Datos Multicanal y Display OLED I2C

Universidad Nacional de Colombia
Electrónica Digital 2016684 — 2026-1
Prof. Ricardo Amézquita Orozco

Información General

Campo	Detalle
Sesión	#6 — Miércoles 25 de Marzo, 2026
Tema	Adquisición de Datos Multicanal y Display OLED I2C
Duración estimada	240 minutos

Materiales Necesarios

Hardware

Componente	Cant.	Descripción	Disp.
Arduino Uno	1		✓
Display OLED SSD1306 128×32	1	Módulo I2C, dirección típica 0x3C	
LDR	1	Fotorresistencia	✓
Sensor LM35	1	Temperatura, 10 mV/°C	✓
Potenciómetro 10 kΩ	1		✓
Resistencia 10 kΩ	2	Divisor LDR + pull-down botón	✓
Botón pulsador	1	Normalmente abierto	✓
Protoboard	1		✓
Cables jumper	Varios	Macho-macho	✓
Cable USB	1		✓

✓ = componente disponible de sesiones anteriores.

Software

- Arduino IDE 1.8.x o 2.x
- Librería **Adafruit SSD1306** — instalar desde Library Manager del Arduino IDE (seleccionar "Install all" para incluir automáticamente Adafruit GFX)
- Python 3 con librería pyserial (pip install pyserial)

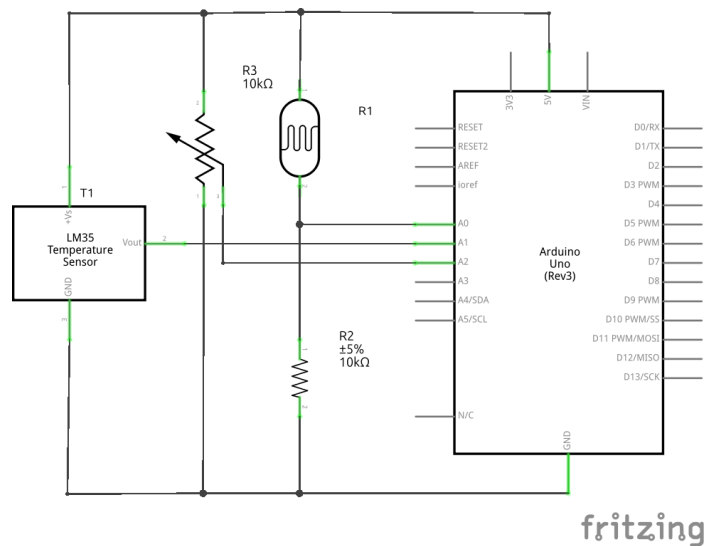
Actividades

Tiempo estimado: Act. 1 (~40 min) · Act. 2 (~25 min) · Act. 3 (~60 min)

Actividad 1: Adquisición multicanal con captura Python y hoja electrónica

Objetivo: Ejecutar el flujo completo de adquisición de datos — Arduino → Serial → Python → archivo CSV → hoja electrónica — y confirmar que los tres canales analógicos operan correctamente.

Circuito



Notas de pinout:

- **LM35 (TO-92):** Orientar con la cara plana hacia arriba. Pin izquierdo (V+) a 5V; pin central (OUT) a A1; pin derecho (GND) a GND.
- **Pines A4 y A5 reservados:** Desde que se invoca `Wire.begin()`, estos pines son controlados por el periférico TWI del ATmega328P. No conectar sensores analógicos a A4 ni A5.

Código

Archivo: lab-06-partel-multicanal-csv.ino — código completo: muestreo periódico de tres canales con `millis()` y emisión CSV por Serial a 9600 baudios.

Script Python provisto — no es necesario escribirlo. El archivo se encuentra junto al .ino.

Procedimiento

Montar el circuito con los tres sensores analógicos: LDR en A0 (con resistencia de 10 kΩ entre A0 y GND como divisor de voltaje), LM35 en A1 (salida directa), y potenciómetro en A2 (wiper; extremos a 5V y GND). Cargar lab-06-partel-multicanal-csv.ino en el Arduino.

Cerrar el Serial Monitor del Arduino IDE. Ejecutar el script Python provisto. Confirmar que la consola muestra el encabezado `t_ms, raw_ldr, raw_lm35, raw_pot` seguido de líneas de datos con cuatro valores separados por comas.

Durante la captura (~60 s), estimular cada sensor:

- Cubrir y descubrir el LDR con el dedo.
- Sostener el LM35 entre los dedos durante 15–20 s.
- Girar el potenciómetro de un extremo al otro.

Presionar `Ctrl+C` para detener la captura. El script indicará el número de filas capturadas y el nombre del archivo .csv generado.

Abrir el archivo .csv en Excel o Google Sheets. Generar una **gráfica de dispersión** de raw_ldr vs t_ms (eje X: tiempo en ms; eje Y: valor raw del LDR). Calcular **promedio, mínimo y máximo** de cada una de las tres columnas de datos (raw_ldr, raw_lm35, raw_pot) usando fórmulas de la hoja de cálculo.

Datos

Captura requerida: Primeras y últimas 5 líneas del archivo .csv generado, mostrando el encabezado. Indicar el conteo total de filas y el tiempo de captura (~60 s).

Gráfica requerida: raw_ldr vs t_ms a partir del archivo CSV. Eje X: t_ms (tiempo en ms); Eje Y: raw_ldr (valor ADC 0–1023). Debe demostrar: los picos y valles correspondientes a los momentos en que el LDR fue cubierto y descubierto durante la captura.

Tabla 1 — Estadísticas por canal

Canal	Promedio	Mínimo	Máximo
LDR (raw_ldr)			
LM35 (raw_lm35)			
Potenciómetro (raw_pot)			

Los valores deben calcularse con fórmulas de la hoja electrónica, no copiarse manualmente.

Pregunta de análisis: A partir de los valores de la Tabla 1, calcule la temperatura ambiente aproximada usando la fórmula $\text{tempC} = \text{rawLM35} \times 5.0 / 1023.0 / 0.01$. ¿El resultado es coherente con la temperatura esperada (~20–25 °C)?

Pregunta de análisis: Calcule la cadencia real de muestreo como el promedio de las diferencias entre timestamps consecutivos (t_ms) en el archivo CSV. Compare con el valor nominal de 500 ms y explique cualquier diferencia observada.

Actividad 2: Display OLED I2C — Scanner y vista de un sensor

Objetivo: Ejecutar el I2C Scanner para verificar la dirección del display OLED SSD1306 y operar la visualización de un sensor en pantalla.

Código

Archivo: lab-06-parte2-oled-basico.ino — código completo: I2C Scanner en setup() + muestreo periódico del potenciómetro y visualización en OLED en loop().

Procedimiento

Conectar el módulo OLED al bus I2C del circuito existente: SDA → A4, SCL → A5, VCC → 5V, GND → GND. Los sensores analógicos de la Actividad 1 permanecen conectados. Cargar el sketch.

Abrir el Serial Monitor a 9600 baudios y verificar la salida del I2C Scanner: debe reportar un dispositivo en la dirección 0x3C. Si detecta una dirección diferente (por ejemplo 0x3D), modificar la constante ADDR_OLED en el sketch y recompilar.

Confirmar que el OLED muestra el valor raw del potenciómetro (Pot: 512). Girar el potenciómetro de un extremo al otro y verificar que el valor en pantalla cambia suavemente entre ~0 y ~1023, con una latencia de aproximadamente 500 ms.

Datos

Tabla 2 — Verificación del sistema I2C

Elemento verificado	Resultado
Dirección I2C detectada por el Scanner	
Texto mostrado en el OLED (transcribir)	
¿El valor cambia al girar el potenciómetro? Sí / No	

Captura requerida: Serial Monitor mostrando la salida del I2C Scanner con la dirección detectada.

Pregunta de análisis: ¿Por qué no es posible conectar un sensor analógico a A4 o A5 mientras el bus I2C está activo?

Actividad 3: Integración con cuatro pantallas conmutables — general y detalle por sensor

Objetivo: Integrar adquisición multicanal, display OLED y logging CSV en un sistema cohesivo con cuatro pantallas conmutables por botón, gestionando estado, variables acumuladoras de mínimo, máximo y promedio por canal, y coordinación de subsistemas sin bloqueo.

Código

Sin archivo suministrado. Escribir desde cero integrando el código de las Actividades 1 y 2 con los requisitos funcionales descritos a continuación.

Circuito

Agregar el botón conmutador al circuito existente: un terminal a D7, el otro a 5V, con resistencia pull-down de 10 kΩ entre D7 y GND.

Procedimiento

Implementar un sketch que cumpla los siguientes requisitos:

Muestreo y CSV continuo: Muestrear los tres canales (LDR en A0, LM35 en A1, potenciómetro en A2) cada 500 ms usando `millis()`. Emitir una línea CSV por Serial con formato `t_ms,raw_ldr,raw_lm35,raw_pot`. La emisión no se interrumpe al cambiar de pantalla.

Conversión a unidades físicas: Convertir los valores raw a unidades físicas para mostrar en el OLED:

Canal	Fórmula	Unidad
LDR	$\text{valorRelativo} = \text{rawLDR} \times 100.0 / 1023.0$	% (valor relativo de iluminación 0-100%)
LM35	$\text{tempC} = \text{rawLM35} \times 5.0 / 1023.0 / 0.01$	°C
Potenciómetro	$\text{porcentaje} = \text{rawPot} \times 100.0 / 1023.0$	% (posición angular 0-100%)

Cuatro pantallas conmutables por botón (ciclo simple): El botón en D7 avanza cíclicamente entre cuatro pantallas. Cada pulsación (flanco ascendente LOW→HIGH con debouncing de 50 ms) incrementa la pantalla con $\text{pantalla} = (\text{pantalla} + 1) \% 4$.

Las cuatro pantallas son:

- **Pantalla 0 — General** (3 líneas, 3 valores en unidades físicas):

LDR: 45%
LM35: 23C
Pot: 50%

- **Pantalla 1 — Detalle LDR** (4 líneas, llenan los 32 px del display):

LDR: 45%
Mn: 12%
Mx: 89%
Av: 52%

- **Pantalla 2 — Detalle LM35** (4 líneas, formato idéntico en °C):

LM35: 23.5C
Mn: 21.0C
Mx: 28.3C
Av: 23.1C

- **Pantalla 3 — Detalle Pot** (4 líneas, formato idéntico en %):

Pot: 50%
Mn: 0%
Mx: 100%
Av: 48%

Tracking de estadísticas acumuladas: Mantener mínimo, máximo y promedio por canal desde el inicio de la operación. Inicializar $\text{minCanal}[i] = 1023$, $\text{maxCanal}[i] = 0$ y $\text{sumaCanal}[i] = 0$ en `setup()`. En cada ciclo de muestreo, actualizar mínimo y máximo con los valores raw. El promedio se calcula al momento de mostrar como $\text{sumaCanal}[i] / \text{nMuestras}$, convertido a unidades físicas.

Restricciones de código:

- No usar objetos String.
- Usar `F()` en todos los literales de texto de `Serial.print()` y `display.print()`.
- No usar `delay()`. Todo el timing debe basarse en `millis()`.
- Llamar `display.display()` una sola vez por ciclo de muestreo, no en cada iteración del `loop()`.

Pista: Estructura de variables globales sugerida:

```

byte pantalla = 0; // Pantalla activa (0-3)
int estadoAnteriorBoton = LOW; // Para detección de flanco
unsigned long tiempoUltimoFlanco = 0; // Para debouncing del botón
float sumaCanal[3] = {0, 0, 0}; // Suma acumulada de valores raw por canal
int minCanal[3] = {1023, 1023, 1023}; // Mínimo por canal (inicializar a valor máximo)
int maxCanal[3] = {0, 0, 0}; // Máximo por canal (inicializar a valor mínimo)
unsigned long nMuestras = 0; // Contador de muestras

```

Conmutación: comparar estado actual del botón con estadoAnteriorBoton. Si hay transición LOW→HIGH y han pasado al menos 50 ms desde tiempoUltimoFlanco, avanzar pantalla = (pantalla + 1) % 4 y actualizar tiempoUltimoFlanco.

El promedio para LDR y potenciómetro se calcula como $(\text{int})(\text{sumaCanal}[i] / \text{nMuestras} \times 100.0 / 1023.0)$. Para LM35: $(\text{sumaCanal}[i] / \text{nMuestras}) \times 5.0 / 1023.0 / 0.01$.

Para mostrar la temperatura con un decimal en las pantallas de detalle (Pantalla 2), use `display.print(tempC, 1)`. En la vista general (Pantalla 0), use `display.print((int)tempC)` para mostrar sin decimales.

Tiempo estimado: 60 minutos.

Datos

Captura requerida: Serial Monitor mostrando el CSV emitiéndose sin interrupción mientras se presiona el botón para cambiar de pantalla. Verificar que no hay gaps ni líneas incompletas durante las conmutaciones.

Foto requerida: Montaje con el OLED mostrando la Pantalla 0 (General, 3 líneas). Etiquetar cada valor indicando canal y unidad.

Foto requerida: Montaje con el OLED mostrando una de las pantallas de detalle (Pantalla 1, 2 o 3, 4 líneas). Etiquetar las cuatro líneas (valor actual, mínimo, máximo, promedio). Indicar a qué canal corresponde.

Pregunta de análisis: El ATmega328P tiene 2048 bytes de SRAM. El buffer del display OLED ocupa 512 bytes. Estime el consumo de SRAM de las variables globales del sketch (arrays de estadísticas, contadores, flags). ¿Cuánta SRAM queda disponible para stack y variables locales?

Pregunta de análisis: Durante la conmutación de pantallas, el CSV continúa emitiéndose sin interrupción. Identifique qué mecanismos del código garantizan que la emisión CSV, la actualización del OLED y la lectura del botón son tareas independientes que no se bloquean mutuamente.

Pregunta de análisis: El debouncing del botón usa una ventana de 50 ms con `millis()`. ¿Por qué no es viable usar `delay(50)` para este propósito en un sistema que debe muestrear sensores cada 500 ms y actualizar el OLED? Proponga un valor de ventana de debouncing inadecuado para este sistema y justifique su respuesta.

Análisis para el Informe

Preguntas de Análisis

1. En el formato CSV de este laboratorio, el timestamp `t_ms` es el valor de `millis()` en el momento del muestreo, no el tiempo real de reloj (hora del día). ¿Qué información se pierde con este enfoque? ¿Cómo podría modificarse el script Python de la Actividad 1 para que cada línea del archivo incluya un timestamp de tiempo real además del `t_ms` del Arduino?
2. Compare la cadencia de muestreo medida en la Actividad 1 con la observada durante la Actividad 3. ¿Agregar el manejo del OLED y el botón afecta la regularidad del intervalo de muestreo? Justifique su respuesta con datos de las capturas de Serial Monitor.

Pregunta Abierta

El sistema de la Actividad 3 mantiene estadísticas acumuladas (mínimo, máximo, promedio) desde el inicio de la operación. Proponga una modificación para que las estadísticas se calculen sobre una ventana deslizante de las últimas N muestras en lugar del total acumulado. ¿Qué estructura de datos utilizaría? ¿Qué restricciones impone la SRAM de 2 KB del Arduino Uno sobre el tamaño máximo de N? Considere que cada muestra consiste en tres valores raw de 10 bits.

Fecha límite de entrega: Miércoles 8 de abril de 2026, 23:59 (Informe Bloque 2: S4, S5, S6).

Troubleshooting

Síntoma	Causa probable
OLED apagado o muestra píxeles aleatorios	SDA/SCL invertidos; VCC/GND invertidos; dirección I2C $\neq$ 0x3C
I2C Scanner no detecta dispositivos	Conexión ausente de SDA, SCL o VCC del OLED
OLED se congela o comportamiento errático en Act. 3	Uso de objetos <code>String</code> que fragmentan la heap; <code>display.display()</code> en bucle
Botón provoca múltiples cambios de pantalla por pulsación	Rebote mecánico sin debouncing, o detección por nivel en lugar de transición
Script Python no detecta el puerto serial	Puerto ocupado por Serial Monitor; nombre de puerto incorrecto
CSV capturado por Python tiene filas incompletas	Buffer serial lleno; lectura interrumpida antes del terminador <code>\n</code>
Lecturas del LM35 inestables o irreales	Alimentación USB inestable ($V_{ref} \neq 5.00V$); ruido en la línea analógica